

ИИ-агенты в 2026



Влад Прошинский
AI-консультант, ex-CPO

Обо мне

Влад Прошинский

AI-консультант, ex-CPO

- 10 лет в IT. Запускал IT продукты в Ozon, SOKOLOV, Деловые Линии, RUTUBE, Газпромбанке
- Сейчас помогаю компаниям внедрять AI так по отработанной методологии (Yakov & Partners Playbook) так, чтобы это давало измеримый результат.

Специализация:

- ИИ в маркетинге
- ИИ для отделов продаж
- Автоматизация бизнес-процессов



Канал «ИИ для продакта & CPO»
<https://t.me/AImademyday>

Программа

- Чем AI-агент отличается от чат-бота и «просто LLM».
- Выбор моделей: качество, стоимость, контекст, latency, требования к вызову инструментов.
- Где агенты дают ROI: браузер/десктоп, API-оркестрация, операционная рутина.
- Бенчмарки: LMArena (чат), SWE-bench (код), Open LLM Leaderboard (open), AgentBench (агентность).

Результат: умеете запускать локальные модели (Llama, Mistral) через Ollama/vLLM и выбирать режимы/инструменты под задачу

Разберем

1

Чат-бот vs LLM vs агент

Определения и ключевые отличия

2

Анатомия агента

LLM + tools + memory + RAG + MCP

3

Agent vs Workflow

Когда что использовать

4

Выбор модели

Качество, цена, контекст, latency, tool use

5

Бенчмарки

LMarena, SWE-bench, Open LLM, AgentBench

6

Локальный запуск

Установка и запуск Qwen3 из Ollama

ЧАСТЬ

1

**Чат-бот vs LLM vs агент
что это вообще такое**

Почему 2026 — год агентов

40%

корпоративных приложений
встроят task-specific AI-агентов
к концу 2026

Gartner

8×

рост за год — в 2025 было <5%

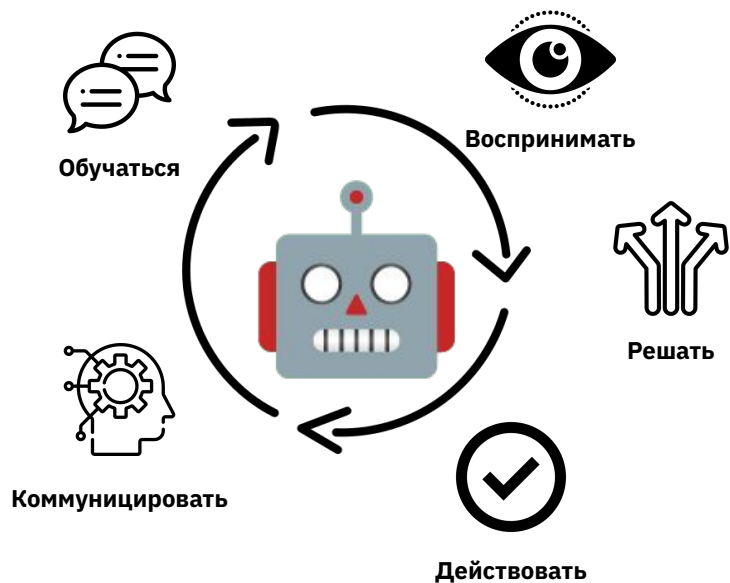
Gartner

\$80B

экономии на контактных
центрах к 2026

Gartner 2024

Что могут делать AI-агенты?



Воспринимать: собирать информацию из своей среды

Принимать решения: оценивать варианты и выбирать лучший для достижения целей

Действовать: выполнять задачи или команды для достижения результатов

Коммуницировать: взаимодействовать с системами, пользователями и другими агентами

Обучаться: со временем улучшаться, анализируя действия и результаты

Их часто путают

Чат-бот

Reactive. Отвечает на промпт и замолкает. Rule-based или LLM-powered. Не выходит за границы диалога.

LLM (+ RAG)

Генерирует контент, рассуждает, ищет. Даёт ответ, но не выполняет действия в системах.

AI-агент

Получает цель, декомпозирует, выбирает инструменты, исполняет, корректирует ошибки. Stateful. Multi-step.

Архитектура ИИ-агентов

Анализ требований и
приоритизация



Дизайн и подготовка
прототипа



Тестирование и
выход в пром



Работа с рисками



Разработка



AI-оркестратор: координация, постановка задач, коммуникация с человеком.

AI-агенты: исследователь, аналитик, приоритизатор, бизнес-аналитик, продуктовый аналитик.

Коммуникация: протоколы (MCP, LangChain) для бесшовного взаимодействия, API краулеры и парсеры

Цикл: ввод → планирование → действие → обратная связь → обновление памяти.

Reactive loop vs Reasoning loop

- Чат-бот: trigger → LLM-ответ → конец. Нет памяти о цели после закрытия чата.
- Агент (ReAct паттерн): Goal → Observation → Reasoning → Action → Feedback → Correction → repeat
- Цикл повторяется 5–200+ раз, пока цель не достигнута
- Агент получает ground truth (эталонные данные) из окружения на каждом шаге (tool results, code output)
- Может приостановиться на checkpoint и запросить помощь человека

Пример: обработка возврата в e-commerce

Чат-бот

Отвечает на вопросы про условия возврата, часы работы, контакты.

LLM + RAG

Пишет персонализированное письмо-инструкцию конкретному клиенту в реальном времени.

AI-агент

Eligibility check → pickup → refund → inventory → confirmation. Полный цикл без человека.

Уровни зрелости

Уровень 1 — информация.

Уровень 2 — контент.

Уровень 3 — действие и результат.

Триггер выбора: когда нужен именно агент

- Процесс занимает 3+ шагов или касается >1 системы — чат-бот упадёт
- Нужна адаптация к новому контексту (UI поменялся, данные изменились)
- Исход не предопределён: нужно принимать решения на ходу
- Результат проверяется автоматически (тесты, API-ответы, state изменился)
- Инвестиция в агент окупается объёмом и сложностью задач

ЧАСТЬ

2

**Анатомия агента
из чего состоит**

Что внутри агента

1. LLM brain

Reasoning + planning. Мозг принимает решения что делать дальше.

2. Tools / Function calling

Руки агента: API, shell, browser, DB-запросы, вычисления.

3. Memory

Short-term (контекст сессии) + long-term (vector DB, файлы, прошлые traj).

4. Knowledge base

RAG: справочники, документация, данные компании под контроль галлюцинаций.

LLM brain: критичные способности для агента

- Instruction following — пойти по инструкции, не «придумать похожее»
- Long-term reasoning — держать цель на 30+ шагах, не терять контекст
- Tool calling accuracy — правильно выбрать инструмент и заполнить аргументы
- Error recovery — увидеть, что шаг упал, переосмыслить, попробовать иначе
- Poor long-term reasoning и decision-making — главная причина провала агента (AgentBench)

Tools: руки агента

- Function calling (OpenAI, Anthropic, Gemini): модель возвращает JSON с именем функции и аргументами
- Типы: API-вызовы, shell/bash, file read/write, DB-запросы, вычисления, web-поиск
- Спецификация через JSON Schema: name, description, parameters

Важно:

- хороший tool description решает больше, чем выбор модели
- Anti-pattern: 50+ tools в одном агенте — падает accuracy. Дробите на subagents

Memory: 4 вида памяти агента

Context window

То, что в промпте сейчас. Быстро, но ограничено (128K–1M токенов).

Conversation history

Предыдущие turns текущей сессии — обрезаются при переполнении контекста.

Long-term memory

Vector DB (Qdrant, Pinecone, pgvector) — факты, предпочтения, долгая память.

Episodic memory

Прошлые успешные/неуспешные логи как справочник для будущих задач.

RAG: Knowledge base

- LLM знает мир, но не знает вашу компанию, продукт, политики
- RAG (Retrieval-Augmented Generation): запрос → поиск в БД → inject в контекст → генерация
- Альтернативы: CAG (Cache-Augmented), Agentic RAG (агент сам ищет итеративно)
- Правило: качество retrieval критичнее качества LLM — мусор на входе = мусор на выходе
- Для агента: knowledge base + tools под одним интерфейсом = MCP

Примеры ценного контекста для ИИ

- Сайт вашей компании/продукта (используйте браузер для «сохранить как PDF» / кроулер)
- Презентация стратегии вашей компании (экспорт в формате PDF)
- Все ваши данные по сегментации и исследованию клиентов
- Исследование вашей конкурентной среды
- Видение компании, стратегия, квартальные плановые документы и т. д.
- Корпоративные процессы
- Организационная структура и роли вашей команды
- Прошлые ретроспективы вашей команды
- Ваша последняя оценка производительности или неформальная обратная связь от менеджера

MCP: Model Context Protocol

- Открытый протокол от Anthropic (ноя 2024), де-факто стандарт в 2025–2026
- Единый интерфейс между LLM и внешним миром: tools + resources + prompts
- Готовые серверы: Gmail, Google Drive, Slack, Linear, Notion, GitHub, Calendar, n8n — десятки
- Любая LLM-среда (Claude, Cursor, VS Code, ChatGPT) подключает любой MCP-сервер
- Принцип: пишете сервер один раз — работает со всеми моделями. USB-C для AI-агентов.

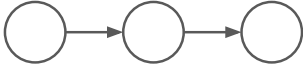
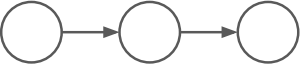
ЧАСТЬ

3

Agent vs Agentic Workflow

КОГДА ЧТО ИСПОЛЬЗОВАТЬ

Автоматизация VS AI workflow VS AI agent

	Автоматизация	AI workflow	AI agent
Определение	 Программа, которая выполняет заранее определенные, основанные на правилах задачи автоматически.	 Программа, которая вызывает LLM через API для одного или нескольких шагов.	 Программа, предназначенная для выполнения недетерминированных задач автономно.
Основы	Булева логика	Булева логика + нечеткая логика	Нечеткая логика + автономность
Типы задач	Детерминированные, предопределённые задачи	Детерминированные задачи, требующие гибкости	Недетерминированные, адаптивные задачи
Сильные стороны	- Надёжные результаты - Быстрое выполнение	- Лучше справляется со сложными правилами - Отлично подходит для распознавания шаблонов	- Высокая адаптивность к новым переменным - Имитация человеческого поведения и рассуждений
Слабые стороны	- Ограничена только явно запрограммированными задачами - Не может адаптироваться к новым сценариям - Сложности с обработкой комплексности	- Требует данных для эффективного обучения моделей - Труднее отлаживать и интерпретировать	- Менее надёжен, может давать непредсказуемые или нежелательные результаты - Медленнее выполняет задачи
Примеры	Отправка уведомления в Slack каждый раз, когда на сайте регистрируется новый лид	Анализ, скрининг и маршрутизация каждого входящего лида с сайта с помощью ChatGPT	Выполнение полноценного интернет-поиска по каждому входящему лиду и обновление информации

Определение от Anthropic — берём как рабочее

Agentic Workflow

Система, где LLM и tools оркестрируются через предопределённый code path. Разработчик пишет шаги: step1 → step2 → step3. LLM — часть потока, но не управляет им.

AI Agent

Система, где LLM сама динамически управляет процессом и выбором инструментов. Разработчик задаёт цель и tools — агент решает последовательность.

Когда использовать что

Workflow

Задача понятна, сценариев мало, цена ошибки высокая, нужна предсказуемость и аудит.

Workflow

Обработка входящих заявок: classify → route → generate reply → send → log.

Agent

Открытая задача, много путей решения, нужна гибкость, решения зависят от контекста.

Agent

Deep research: план → поиск → чтение → декомпозиция → новый поиск → финальный отчёт.

Правила

- Начинайте с самого простого решения — один LLM-call + RAG часто достаточно
- Сложнее → workflow: предсказуемость для хорошо-определенных задач
- Ещё сложнее → agent: когда нужна гибкость и model-driven решения в масштабе
- Agentic системы влияют на latency и косты на performance (выбор должен быть оправдан)
- Не все задачи нужно агентизировать — 80% закрываются workflow

ЧАСТЬ

4

Выбор модели для агента

4 критерия выбора модели

Качество

Benchmark score + evals на вашем домене.
Bench $\neq$ прод.

Стоимость

\$/1M токенов input+output. Агенты делают десятки вызовов — цена умножается.

Контекст

128K / 200K / 1M. Больше контекст — больше забываний, и не бесплатно.

Latency + Tool use

Reasoning models дают качество за время.
Агентам нужны надёжные tool calls.

Качество = под задачу, а не в целом

- Нет одной «лучшей» модели. Claude Opus 4.7 — топ SWE-bench (87.6%), Gemini 3 Pro — топ long context, GPT-5.4 — топ computer use
- Для агентов критичны: instruction following, tool calling accuracy, error recovery
- Reasoning modes (Claude Thinking, GPT reasoning) — плюс 5–10% качества, минус 2–5× по latency и цене

Не берите флагман на каждую задачу.

Используйте router: более дешевую модель для классификации + фронтир для мозгов + исполнителя процессов подбирайте по параметрам

Цена фронтирных моделей за 1М токенов

\$5/\$25

Claude Opus 4.6/4.7 — флагман
Anthropic (input / output)

1М контекст без доплаты

\$3/\$15

Claude Sonnet 4.6, Gemini 3 Pro
— в 5× дешевле Opus

основная рабочая лошадка

\$1/\$5

Claude Haiku 4.5, GPT-5 Nano —
для routing и простых шагов

в 25× дешевле флагмана

Контекст: 1М стало нормой

- Все топовые модели 2026 поддерживают 1М токенов контекста
- Claude Opus 4.6/4.7 и Sonnet 4.6 — 1М по стандартной цене (без доплат)
- Gemini 3 Pro — 1М, лучший retrieval (MRCR) на больших контекстах
- Свыше 200K input у многих моделей — long-context тариф $\times 1.5-2$
- Оптимизация: prompt caching –90% на повторяющиеся system prompts

Latency и tool use — скрытые параметры

- Reasoning модели (Opus Thinking, GPT o3) думают 5–60 сек — плохо для real-time UX
- Для агентских задач $100 \text{ шагов} \times 10 \text{ сек} = 17 \text{ мин}$ на задачу — нужны streaming и progress UI
- Tool use accuracy по TAU-bench: Claude ~69%, GPT-5 ~65%, open models ниже
- Правило: сначала evals на вашем домене, потом production

ЧАСТЬ

5

**Бенчмарки
как читать и чему не
верить**

4 бенчмарка, которые надо понимать

LMarena

Blind human preference, Elo. Как «ощущается» модель в диалоге. Меряет вайб, не задачу.

SWE-bench

500 реальных GitHub issues. Модель должна сгенерировать patch, проходящий тесты.

Open LLM Leaderboard

HuggingFace — сравнение open-weight (Llama, Qwen, Mistral, DeepSeek) на MMLU-Pro, GPQA, MATH.

AgentBench (описание)

8 интерактивных сред: OS, DB, knowledge graph, web shopping и др. Мера agency.

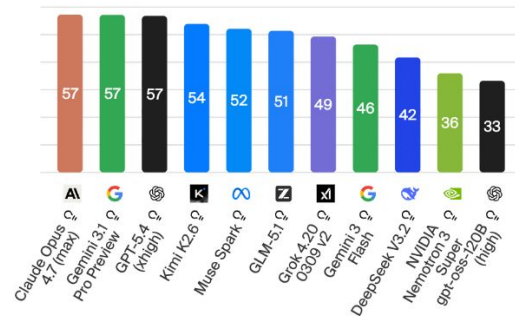
Как выбирать модель для проекта?

Изучать и сравнивать самому: <https://artificialanalysis.ai/> |

Highlights

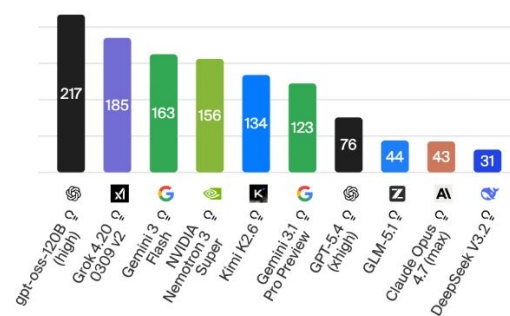
Intelligence

Artificial Analysis Intelligence Index; Higher is better



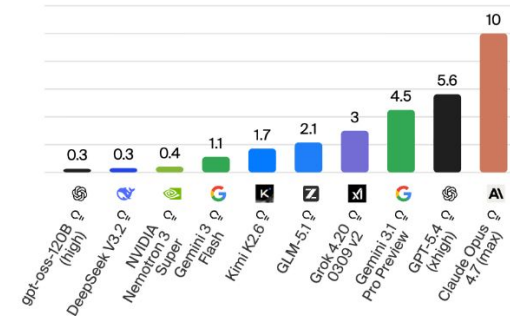
Speed

Output Tokens per Second; Higher is better



Price

USD per 1M Tokens; Lower is better



Как выбирать модель для проекта?

смарт подход — сравнивать в деле

Недавно OpenRouter запустил [Model Fusion](#) 31 марта 2026 года — это экспериментальная функция в разделе OpenRouter Labs, которая отправляет твой промпт сразу нескольким моделям одновременно, анализирует их ответы и объединяет в один лучший результат.

Как это работает

Процесс состоит из трёх шагов:

1. **Query (Запрос)** — твой промпт параллельно отправляется в 3–5+ моделей одновременно
2. **Analyze (Анализ)** — каждый ответ оценивается по точности и глубине
3. **Fuse (Слияние)** — специальная «модель-судья» берёт лучшее из каждого ответа и собирает финальный ответ

Грубо говоря, вместо того чтобы вручную открывать вкладки с GPT, Claude и Gemini и сравнивать — система делает это за тебя и выдаёт уже готовый синтез.

Где смотреть цены провайдеров?

<https://openrouter.ai/models> -
провайдеры

<https://www.mulerouter.ai/> - китайцы

kie.ai - video, image

Выбираем модель для РФ

YandexGPT (Yandex Cloud) — API с оплатой в рублях, хорошо для русскоязычных задач, слабее в code generation и технических задачах на английском

GigaChat API (Сбер) — серверы на территории РФ, SLA в договоре; в декабре 2025 выпустили open-source модель на 702B параметров, лидирует в бенчмарке MERA для русского языка

Сервис	Модели	Оплата из РФ	Наценка	Китайские модели
Bothub	100+	✓ Карты РФ, рубли	~14%	✓ DeepSeek, Qwen, Kimi
Polza.ai	500+	✓ Карты РФ, рубли	~10–25%	✓ DeepSeek, Qwen, Kimi
OfoxAI	50+	Крипто	0–5%	✓ DeepSeek, Qwen, Kimi
proxyapi.ru	~15	✓ Рубли	30–50%	×
aitunnel.ru	~30+	✓ Рубли	25–40%	×
YandexGPT	1	✓ Рубли	—	×
GigaChat API	1	✓ Рубли	—	×

Топ на 21.04

Claude Opus 4.7 – новый флагман для кода и агентов

- Прокачали coding и long-horizon reasoning – модель сама проверяет свои выводы перед тем как отрапортовать
- Новый вижн-модуль (наконец-то, а то была слабее всех) – картинки до 2576px по длинной стороне (в 3 раза больше прежнего), полезно для computer-use агентов и разбора диаграмм
- Лучше следует инструкциям – буквально, до педантичности. Старые промпты под 4.6 могут сломаться, Anthropic прямо предупреждает переписывать
- Появился новый уровень усилий xhigh между high и max – больше контроля над trade-off «думает vs. latency».
- Кажется, max теперь не имеет смысла включать.
- В Claude Code по умолчанию теперь xhigh на всех планах
- Новая команда /ultrareview в Claude Code – выделенная сессия review, ищет баги и design issues. Pro и Max – 3 бесплатных ultrareview в месяц
- Auto mode расширили на Max-план – Claude принимает решения сам, меньше прерываний на permission-запросы

	Opus 4.7	Opus 4.6	GPT-5.4	Gemini 3.1 Pro	Mythos Preview
Agentic coding SWE-bench Pro	64.3%	53.4%	57.7%	54.2%	77.8%
Agentic coding SWE-bench Verified	87.6%	80.8%	—	80.6%	93.9%
Agentic terminal coding Terminal-Bench 2.0	69.4%	65.4%	75.1% self-reported harness	68.5%	82.0%
Multidisciplinary reasoning Humanity's Last Exam	46.9% no tools	40.0% no tools	42.7% no tools (Pro)	44.4% no tools	56.8% no tools
	54.7% with tools	53.3% with tools	58.7% with tools (Pro)	51.4% with tools	64.7% with tools
Agentic search BrowseComp	79.3%	83.7%	89.3% Pro	85.9%	86.9%
Scaled tool use MCP-Atlas	77.3%	75.8%	68.1%	73.9%	—
Agentic computer use OSWorld-Verified	78.0%	72.7%	75.0%	—	79.6%
Agentic financial analysis Finance Agent v1.1	64.4%	60.1%	61.5% Pro	59.7%	—
Cybersecurity vulnerability reproduction CyberGym	73.1%	73.8%	66.3%	—	83.1%
Graduate-level reasoning GPQA Diamond	94.2%	91.3%	94.4% Pro	94.3%	94.6%
Visual reasoning CharXiv Reasoning	82.1% no tools	69.1% no tools	—	—	86.1% no tools
	91.0% with tools	84.7% with tools	—	—	93.2% with tools
Multilingual Q&A MMMLU	91.5%	91.1%	—	92.6%	—

Важный нюанс: Verified vs Pro

(реальность vs маркетинг)

- SWE-bench Verified: 500 задач из публичных репозиториях — модели видели в обучении
- SWE-bench Pro: приватные репозитории, модели не видели — баллы падают в 3×
- Пример: GPT-5 с 74.9% (Verified) до 14.9% (Pro Private)
- Claude Opus 4.5: с 22.7% до 17.8% (Pro)

Вывод: в продакшене на вашем коде реальное качество ближе к Pro, чем к Verified

AgentBench — измеряем agency

8 интерактивных сред: OS, DB, Knowledge Graph, Card Game, Lateral Thinking, House-holding, Web Shopping, Web Browsing

Измеряет: long-term reasoning, decision-making, instruction following

Вывод: gap между top commercial и open-source значительный на агентских задачах

Главные причины провала: плохое long-term reasoning, decision-making, instruction following

General AgentBench (2026): даже GPT-5 деградирует на 15–30% при смене окружения

AgentBench — измеряем agency

Code-grounded (код и данные)

Среда	Что делает агент	Что проверяется
OS	Работает в Ubuntu bash-терминале: файловые операции, процессы, скрипты	Long-term reasoning, исполнение команд
DB	Пишет SQL-запросы к MySQL: SELECT, INSERT, UPDATE, аналитические цепочки	Multi-step reasoning над структурированными данными
Knowledge Graph (KG)	Ходит по Freebase через инструменты, минимум 5 вызовов инструментов за задачу	Decision-making и планирование в графе знаний

AgentBench — измеряем agency

Game-grounded (игровые среды)

Среда	Что делает агент	Что проверяется
Digital Card Game (DCG)	Играет в пошаговую карточную игру Aquawar против противника	Стратегическое мышление, понимание правил, неполная информация
Lateral Thinking (LTP)	Задаёт да/нет вопросы, чтобы разгадать скрытый сценарий-загадку	Дедукция, творческое мышление, эффективность вопросов
House-Holding (HH)	Управляет персонажем в текстовом симуляторе дома (ALFWorld): «возьми чашку, положи на стол»	Commonsense reasoning, embodied planning

AgentBench — измеряем agency

Lateral Thinking — самая нестандартная среда: агент получает финальную ситуацию («мужчина лежит мёртвым в поле, рядом нераскрытый пакет»), и должен задавая вопросы восстановить скрытую историю. Это чистая дедукция и умение формулировать точные вопросы.

House-Holding основан на ALFWorld — классическом embodied-AI датасете, где агент управляет навигацией по комнатам и предметами только через текст.

Web-grounded (веб)

Среда	Что делает агент	Что проверяется
Web Shopping (WS)	Ищет товары в симулированном интернет-магазине WebShop, сравнивает, покупает нужный	Навигация, понимание описаний, соответствие требованиям
Web Browsing (WB)	Переходит по реальным веб-страницам (Mind2Web), извлекает информацию из динамического контента	Общая веб-навигация, instruction following

Как читать бенчмарки — 5 правил

- Не доверяйте одному — смотрите 3+ бенчмарка под вашу задачу
- Смотрите дату публикации — всё старше 6 мес потеряло актуальность
- Vendor-published scores дают best-case на их scaffold — умножайте на 0.7
- Scaffold (Claude Code, Aider, Codex) меняет результат на 10–20% при одной модели
- Финальное решение — свои evals на 50–200 примерах из вашего домена

ЧАСТЬ

6

Инструменты

Инструменты для ИИ агентов

- n8n (остался в 2025 году, но для части agentic сценариев вполне ОК)
- CoWork Claude (schedule)
- OpenClaw – хайп прошел, куча дыр, дорого
- В топе щас:
 - <https://github.com/paperclipai/paperclip> для оркестрации агентов
 - Hermes Agent (100,000+ github stars) – [видео](#) , как [установить](#)

ЧАСТЬ

7

Локальный запуск Qwen3-4B

Что влезает в MacBook Air M4 (16 GB memory)

Модель	Размер (Q4)	Скорость на M4	Рекомендация
Qwen3-0.6B	~0.5 GB	⚡ Очень быстро	Тесты, edge-задачи
Qwen3-1.7B	~1.2 GB	⚡ Очень быстро	Лёгкие агенты
Phi-4-mini (3.8B)	~2.5 GB	✅ Быстро	Код, reasoning
Qwen3-4B	~2.9 GB	✅ Быстро	Оптимум для агентов
Llama 3.2 3B	~2 GB	✅ Быстро	Универсальный
Qwen3-8B	~5.5 GB	✅ Нормально	Лучшее качество на M4 16GB
Mistral 7B	~4.5 GB	✅ Нормально	Классика для кода
Llama 3.1 8B	~5 GB	✅ Нормально	Хорош для tool-use

Установка Ollama + Qwen3-4B

1. Идем на ollama или в терминал `curl -fsSL https://ollama.com/install.sh | sh`
2. <https://ollama.com/library/qwen3:8b> → `ollama run qwen3:8b`

1. Установить Ollama (macOS)

```
ollama run qwen3:8b
```

2. Запустить сервер (или открыть апп в меню-баре)

```
ollama serve
```

3. устанавливаем размер контекстного окна

```
ollama ps (OLLAMA_CONTEXT_LENGTH=131072 ollama run qwen3:4b — если нужно поднять контекст)
```

Ollama + Gemma/Qwen3.6 + Claude Code

1. Устанавливаем Ollama **curl -fsSL <https://ollama.com/install.sh> | sh**
2. <https://ollama.com/library/qwen3:8b> → ollama run qwen3:8b
3. <https://ollama.com/library/gemma4> → gemma4:e4b (9.6GB)
4. Включаем Gemma в Claude Code ([видео](#))

Установка Ollama + Kimi 2.6

<https://ollama.com/library/kimi-k2.6>

→ ollama run kimi-k2.6:cloud

Claude Code

ollama launch claude --model kimi-k2.6:cloud



Как развернуть самим

Три варианта:

[Dify](#) (125K звёзд на GitHub) — визуальный конструктор. RAG, агенты, аналитика из коробки. Подключаете свои API-ключи, получаете корпоративный ChatGPT за вечер.

[Open WebUI](#) (120K звёзд) — проще некуда. Docker, LDAP-авторизация, работает офлайн. Идеально для параноиков и компаний, где безопасник спит с файрволом под подушкой.

[LobeChat](#) (70K звёзд) — самый красивый интерфейс. 42 провайдера моделей, плагины в один клик. Если сотрудники привыкли к ChatGPT — не заметят разницы.

Что нужно: сервер, Docker, API-ключи провайдеров.

ЧАСТЬ

8

БОНУС

**Пошаговый план
внедрения ИИ агента**

Фаза 1: Оценка

Картографирование задач и процессов:

- Определите все потенциальные задачи и рабочие процессы, которые могут выиграть от автоматизации с помощью агентов
- Классифицируйте задачи как: **Низкоточные** (точность 90%) или **Высокоточные** (требуется почти идеальная точность)
- Приоритизируйте **низкоточные**, но *часто повторяющиеся* задачи для начального внедрения.
- Оцените трудоемкость выполнения задач по сравнению с их стратегической ценностью

Оценка готовности данных:

- Проведите аудит доступности данных для каждого потенциального кейса использования
- Убедитесь, что необходимые API и интеграции доступны
- Оцените качество и полноту данных
- Учитывайте требования к соответствию (compliance) и конфиденциальности

Определение метрик успеха:

- Зафиксируйте текущие метрики производительности (базовые показатели)
- Определите конкретные цели по повышению эффективности (сэкономленное время, обработанный объём)
- Установите метрики качества (точность, уровень ошибок)
- Определите бизнес-метрики влияния (влияние на выручку, удовлетворённость клиентов).

Фаза 2: Внедрение

Начните с малого:

- Выберите один чётко определённый кейс для начального внедрения
- Подготовьте подробную документацию по процессу для выбранного рабочего процесса
- Определите чёткие границы ответственности агента
- Установите конкретные критерии успеха для пилотного проекта

Выбор технологического подхода:

- Оцените варианты без кода (no-code), с низким порогом кодирования (low-code) и кастомной разработки.
- Выберите подходящие ИИ-модели и платформы.
- Учитывайте стабильность вендора и доступность поддержки.
- Оцените масштабируемость решения для будущего расширения.

Проектирование контроля со стороны человека:

- Установите протоколы проверки результатов работы агента.
- Создайте чёткие пути эскалации для исключений и ошибок.
- Определите процессы согласования, где это необходимо.
- Задokumentируйте процессы с участием человека (human-in-the-loop).

Тщательное тестирование:

- Подготовьте тестовые сценарии с использованием исторических данных.
- Проведите симулированные прогоны в контролируемой среде.
- Сравните результаты работы агента с эталонными результатами, полученными вручную.
- Выявите и устраните пробелы в производительности до полного развертывания

Фаза 3: Интеграция

Настройка доступа к данным:

- Настройте безопасные подключения к необходимым источникам данных.
- Установите подходящие механизмы аутентификации.
- Задокументируйте потоки данных и сценарии их использования.
- Реализуйте аудит логов для отслеживания работы с чувствительной информацией.

Интеграция в рабочие процессы:

- Интегрируйте выводы агента в существующие системы.
- Настройте передачу задач между агентами и членами команды.
- Обеспечьте совместимость с текущими инструментами и платформами.
- Минимизируйте вмешательство в существующие процессы.

Проектирование пользовательского взаимодействия:

- Создайте интуитивно понятный интерфейс для взаимодействия человека и агента.
- Обеспечьте видимость действий агента для пользователей.
- Найдите баланс между автоматизацией и контролем со стороны человека.
- Продумайте механизмы обратной связи от пользователей.

Обеспечение безопасности:

- Проанализируйте риски, связанные с доступом агента к данным.
- Реализуйте соответствующие уровни доступа и контроля.
- Задокументируйте протоколы безопасности для работы агента.
- Проведите проверку безопасности перед полноценным внедрением.

Фаза 4: Измерение

Отслеживание показателей эффективности:

- Измерьте сэкономленное время по сравнению с исходными данными (базовыми метриками).
- Отслеживайте объём обработанной работы.
- Рассчитайте стоимость одной транзакции.
- Задокументируйте улучшения в производительности.

Контроль качества:

- Регулярно проводите аудит точности результатов агента.
- Отслеживайте количество и типы ошибок.
- Сравнивайте стабильность и надёжность с результатами, полученными вручную.
- Выявляйте возможности для повышения качества.

Оценка бизнес-результатов:

- Оцените влияние действий агента на выручку.
- Отслеживайте удовлетворённость клиентов в затронутых процессах.
- Измеряйте удовлетворённость сотрудников при взаимодействии с агентом.
- Рассчитайте ROI (окупаемость инвестиций) на основе затрат и полученной выгоды.

Уточнение и итерации:

- Собирайте постоянную обратную связь от пользователей.
- Проводите регулярные ревью (оценки) производительности агента.
- Внедряйте постоянные улучшения в работу агентов и бизнес-процессы.
- Документируйте полученные уроки и выводы для последующих внедрений.